Tabla de Contenido

Objetivo 1
1 Administración de configuración del software 2
1.1 Organización 2
1.2 Responsabilidades 2
1.2.1 Líder de soporte 2
1.2.2 Líder del Proyecto 2
1.2.3 Demás integrantes del equipo 2
1.3 Políticas, directivas y procedimientos 3
2 Actividades de Administración de configuración 3
2.1 Identificación de la configuración 3
2.1.1 Identificación de los elementos de configuración 3
2.1.2 Nombre de los elementos de configuración 3
2.2 Control de Configuración 4
2.2.1 Solicitud de cambios 4
2.2.2 Evaluación de cambios 4
2.2.3 Aprobación o rechazo de cambios 5
2.2.4 Implantación de cambios 6
2.3 Revisiones del estado de las configuraciones 6
3 Programas de Administración de configuración 7
4 Recursos 7
4.3.1 Recursos Humanos 7
4.3.2 Recursos Tecnológicos 8
4.3.3 Recurso Documentales 8
5 Plan de mantenimiento de administración de configuración. 8
5.3.1 Objetivo 8
5.3.2 Alcance 8
5.3.3 Actividades de mantenimiento 9
5.3.4 Frecuencia de mantenimiento 9
5.3.5 Responsables 9
5.3.6 Herramientas utilizadas 10
5.3.7 Control y seguimiento 10
6 Manejo de Codigo 10
6.1 Control de versiones 10
Beneficios: 10
6.2 Estructura de ramas 10
6.3 Flujo de trabajo (Git Flow simplificado) 11
6.4 Manejo de errores 11

Objetivo

Definir los lineamientos, procesos y responsabilidades para la administración de la configuración del software, garantizando el control de seguimiento y trazabilidad de los cambios realizados durante el ciclo de vida del sistema DieselMaps , asegurando la integridad y calidad del producto.

1 Administración de configuración del software

1.1 Organización

El equipo de trabajo estará estructurado de manera que cada integrante tenga roles definidos dentro del proceso de configuración, permitiendo una adecuada gestion de cambios y control de versiones.

1.2 Responsabilidades

1.2.1 Líder de soporte

- Gestionar incidencias y problemas reportados por los usuarios
- Realizar seguimiento a errores detectados
- Coordinar con el equipo tecnico la solucion de fallos
- Verificar que las correciones sean implementadas correctamente y registradas y documentada

1.2.2 Líder del Proyecto

- Supervisar la correcta implementación de la administración de configuración
- Aprobar cambios importantes en el sistema
- Garantizar el cumplimiento de estándares y buenas prácticas
- Coordinar el trabajo del equipo y asignar tareas

1.2.3 Demás integrantes del equipo

- Desarrollar funcionalidades siguiendo las políticas establecidas
- Documentar los cambios realizados
- Usar correctamente el sistema de control de versiones
- Reportar errores o inconsistencias encontradas

1.3 Políticas, directivas y procedimientos

- Todo cambio en el sistema debe ser registrado en el repositorio Git
- Se deben utilizar ramas (branches) para el desarrollo de nuevas funcionalidades
- Ningún cambio puede pasar a producción sin previa validación
- Se debe mantener documentación actualizada de cada versión
- Los commits deben ser claros y descriptivos
- Se deben realizar pruebas antes de integrar cambios al sistema principal
- Se debe mantener respaldo de la base de datos y del código

2 Actividades de Administración de configuración

2.1 Identificación de la configuración

2.1.1 Identificación de los elementos de configuración

- Código fuente.
- Base de datos.
- Documentación.
- Interfaces graficas.
- Apis.
- Scripts.

2.1.2 Nombre de los elementos de configuración

Backend (Java):

- backend_spring_v1.0
- modulo_auth_v1.0
- modulo_estaciones_v1.0
- configuracion_seguridad_v1.0

Frontend (Vite + JavaScript):

- frontend_vite_v1.0

- interfaz_usuario_v1.0
- componentes_ui_v1.0

Base de datos:

- base_datos_v1.0
- modelo_datos_v1.0

Configuración del sistema:

- variables_entorno_v1.0 (.env)
- configuracion_build_v1.0 (vite.config.js, package.json)

2.2 Control de Configuración

Se gestionará el control de configuración mediante el uso de **Git** y repositorios alojados en **GitHub**.
Se definirán ramas principales:

- `main`: versión estable en producción
- backend: integración de funcionalidades y servicios para la funcionalidad
- frontend: incorporación de interfaces de interacción para el usuario

Los artefactos de control: código fuente (frontend en **React**, backend en **Spring Boot**), scripts de base de datos, documentación y configuraciones del sistema.

2.2.1 Solicitud de cambios

Las solicitudes de cambio se registrarán mediante issues en GitHub. Cada solicitud debe incluir:

- Descripción del cambio
- Justificación (problema o mejora)
- Impacto esperado (técnico y de negocio)
- Prioridad (alta, media, baja)

Las solicitudes pueden ser generadas por desarrolladores y demás miembros del equipo de trabajo.

2.2.2 Evaluación de cambios

El equipo técnico (arquitecto y desarrollador) evaluará cada cambio considerando:

- Impacto en la arquitectura (capas, servicios, APIs)
- Riesgos técnicos (rendimiento, seguridad, compatibilidad)
- Complejidad de implementación

- Dependencias con otros módulos (frontend, backend, sensores)

Luego se toma encuentra una chek list para corroborar si el cambio es relevante o no

Ítem	Criterio de evaluación	Estado (✓ / ✗)
1	¿El cambio está claramente definido y documentado?	
2	¿Se entiende el problema o mejora que propone?	
3	¿Se evaluó el impacto en la arquitectura del sistema?	
4	¿Afecta otras funcionalidades existentes?	
5	¿Requiere cambios en base de datos?	
6	¿Implica cambios en la comunicación entre frontend y backend (API)?	
7	¿Se identificaron dependencias con otros módulos?	
8	¿Se estimó el esfuerzo de desarrollo?	
9	¿Se analizaron riesgos técnicos?	
10	¿El cambio es viable dentro del tiempo del proyecto?	

En caso de ser un cambio critico se hará una reunión y se analizará la complejidad y la necesidad de la implementación

2.2.3 Aprobación o rechazo de cambios

El líder de desarollo o arquitecto será responsable de aprobar o rechazar cambios.
Criterios de aprobación:

- Alineación con la arquitectura definida
- Beneficio claro frente al costo
- No comprometer estabilidad del sistema

Los cambios rechazados deberán documentar la razón (ej: alto riesgo, fuera de alcance, mala prioridad).

Ítem	Criterio de implantación	Estado (✓ / ✗)
1	¿Se creó una rama adecuada (Front o Bakend)?	
2	¿El código cumple con los estándares del proyecto?	
3	¿Se implementaron pruebas unitarias?	
4	¿El cambio fue probado localmente?	
5	¿Se realizaron pruebas de integración?	
6	¿No se rompió funcionalidad existente?	
7	¿El frontend y backend funcionan correctamente en	

	conjunto?	
8	¿Se realizó un Pull Request?	
9	¿El cambio fue revisado (code review)?	
10	¿Pasó las validaciones automáticas (build/CI)?	
11	¿Se actualizó la documentación si era necesario?	
12	¿Está listo para desplegarse sin afectar producción?	

2.2.4 Implantación de cambios

Los cambios aprobados se implementarán mediante un proceso de revisión, implementación y prueba

- Los cambios se realizarán en las ramas Frontend o Bakend
- Desarrollo y pruebas unitarias
- Integración mediante Pull Request
- Revisión de código (integridad y coherencia)

2.3 Revisiones del estado de las configuraciones

Las revisiones del estado de configuración tienen como objetivo garantizar la **integridad, trazabilidad y consistencia** de los artefactos del sistema a lo largo del ciclo de vida del proyecto Diesel Maps. Estas revisiones permiten validar que los componentes desplegados corresponden a versiones controladas y aprobadas, evitando inconsistencias entre entornos (desarrollo, pruebas y producción).

Se realizarán revisiones de forma **periódica por sprint** y también antes de cada despliegue a producción. Durante estas revisiones se verificará:

- La correcta sincronización entre los repositorios en **Git** y los entornos desplegados.
- La coherencia entre versiones del frontend en **React** y el backend en **Spring Boot**.
- La correspondencia entre cambios implementados y solicitudes aprobadas (trazabilidad).
- El estado de las ramas principales (Frontend, Bakend) y su estabilidad.
- La integridad de configuraciones críticas (variables de entorno, conexiones a base de datos, endpoints).

Estas revisiones permiten detectar desviaciones tempranas, reducir riesgos en despliegues y asegurar que el sistema mantiene alineación con la arquitectura definida.

Criterio	Estado (✓ Aceptado / ✕ Rechazado)
Versiones en repositorio coinciden con entorno desplegado	
Frontend y backend son compatibles entre sí	
Cambios implementados están aprobados y documentados	
Ramas principales (`main`, `develop`) están estables	
Configuraciones críticas (env, DB, API) son correctas	

3 Programas de Administración de configuración

Para garantizar el control y seguimiento de los cambios en el sistema, se utilizan las siguientes herramientas:

- **Git**
  Sistema de control de versiones utilizado para gestionar los cambios en el código fuente del backend y frontend, permitiendo el trabajo colaborativo y la trazabilidad de modificaciones.
- **GitHub**
  Plataforma utilizada para alojar el repositorio del proyecto, facilitando la gestión de versiones, control de cambios y colaboración entre los integrantes del equipo.
- **Visual Studio Code**
  Entorno de desarrollo utilizado para la edición del código y la integración con herramientas de control de versiones.
- **Node.js**
  Utilizado para la gestión de dependencias y ejecución del entorno de desarrollo del frontend.
- **Vite**
  Herramienta utilizada para la construcción, ejecución y optimización del frontend.

4 Recursos

Para llevar a cabo la administración de configuración del software se requieren los siguientes recursos:

4.1.1 Recursos Humanos

- **Líder del proyecto:** supervisa la gestión de cambios y aprueba versiones.
- **Desarrollador y arquitecto:** realizan modificaciones en backend y frontend.

- **Tester :** verifica que los cambios no afecten el funcionamiento del sistema**.**

4.1.2 Recursos Tecnológicos

- **Computadores de desarrollo** con capacidad para ejecutar el entorno del proyecto.
- **Conexión a Internet** para sincronización con repositorios remotos.
- **Repositorio en GitHub** para almacenamiento del código.
- **Sistema de control de versiones Git** para registrar cambios.
- **Visual Studio Code** como editor principal.
- **Node.js y Vite** para ejecución del frontend.
- **Base de datos** para pruebas del sistema.

4.1.3 Recurso Documentales

- Plan de administración de configuración.
- Historial de versiones.
- Registro de incidencias.
- Manuales técnicos del sistema.
- Documentación del código fuente.

5 Plan de mantenimiento de administración de configuración.

5.1.1 Objetivo

Establecer las actividades necesarias para mantener actualizado, controlado y trazable el sistema, garantizando la correcta gestión de versiones, cambios y elementos de configuración a lo largo del tiempo.

5.1.2 Alcance

Aplica a todos los componentes del sistema, incluyendo:

- Código fuente (frontend y backend)
- Base de datos
- Documentación técnica del codigo

5.1.3 Actividades de mantenimiento

Control de versiones

- Registrar todos los cambios en el repositorio Git.
- Uso de ramas (branching) para nuevas funcionalidades y correcciones.
- Integración mediante pull requests.

Gestión de cambios

- Todo cambio debe ser documentado.
- Validación previa antes de integrar a la rama principal.
- Aprobación por el líder del desarrollo o arquitectura.

Actualización de documentación

- Mantener actualizado el historial de versiones.
- Registrar cambios en el sistema.
- Documentar nuevas funcionalidades.

Respaldo de información

- Realizar copias de seguridad del repositorio y base de datos.
- : semanal o antes de cambios importantes.

Control de configuración

- Identificar versiones estables del sistema.
- Etiquetar versiones (releases).
- Mantener consistencia entre entornos (desarrollo, pruebas, producción).

5.1.4 Frecuencia de mantenimiento

- Control de versiones: continuo
- Actualización de documentación: en cada cambio
- Respaldos: realizar un respaldo minimo semanal para llevar un mejor control.
- Revisión de configuración: mensual o por versión

5.1.5 Responsables

- **Líder de calidad:** controla versiones y cambios

- **Desarrollador y arquitecto:** implementan cambios
- **Líder de calidad:** aprueba cambios

5.1.6 Herramientas utilizadas

- Git (control de versiones)
- GitHub (repositorio)
- Visual Studio Code (desarrollo)
- Node.js y Vite (frontend)

5.1.7 Control y seguimiento

- Uso de commits descriptivos
- Registro de incidencias
- Revisión de cambios (code review)
- Monitoreo de versiones

6 Manejo de Codigo

Manejo de código tiene como objetivo establecer las prácticas, herramientas y procedimientos necesarios para garantizar la correcta gestión, control, mantenimiento y evolución del código fuente del sistema DieselMaps.

Esto permite asegurar la calidad del software, facilitar el trabajo colaborativo y reducir errores durante el desarrollo.

6.1 Control de versiones

Para el control del código fuente se utilizará un sistema de control de versiones distribuido.

Se recomienda el uso de:

- Git
- Plataforma de repositorio como GitHub

Beneficios:

- Registro de cambios en el código
- Trabajo colaborativo sin conflictos
- Recuperación de versiones anteriores
- Seguimiento de errores

6.2 *Estructura de ramas*

Se define la siguiente estrategia de ramas:

- **main** → versión estable del sistema
- **Backend** → Se maneja nuevas versiones de backend
- **Frontend/** → Se maneja nuevas versiones de frontend
- **hotfix/** → corrección de errores críticos

6.3 *Flujo de trabajo (Git Flow simplificado)*

1. Desarrollar la funcionalidad
2. Realizar commits con mensajes claros
3. Subir cambios al repositorio
4. Crear Pull Request
5. Revisar código

6.4 *Manejo de errores*

6. Registrar errores encontrados
7. Documentar soluciones
8. Evitar repetir fallos

CONTROL DE CAMBIOS		
Fecha	**Descripción**	**Autor(es)**
21/04/2026	LS – DANIEL RODRIGUEZ	DANIEL RODRIGUEZ